Using a Sandboxed Ticketing Bot to Teach Digital Trust and IT Audit*

Shantel Deleon[a], Lorraine Lee[b], Hagit Levy-Shalev[a], and Hunter Ng[a]

[a] *Baruch College, City University of New York, United States*
[b] *University of North Carolina Wilmington*

August 21, 2026

Abstract

The verification of Information Technology (IT) controls is a core responsibility of IT auditors. This case places students in the role of IT auditors assigned to evaluate weak controls in a sandboxed online ticketing system. Students first participate in limited ticket drops as ordinary users, then examine how the website communicates with the backend server and observe how a simple bot program can use an easily discoverable API request to automate ticket purchases. The case uses a controlled setting to show how predictable transaction requests and insufficient backend restrictions can expose weaknesses in purchase limits, transaction authorization, logging, exception reporting, and monitoring. Students document their observations through a website transaction discovery worksheet, ticket-drop observations, an automation observation sheet, an internal-control review matrix, and a short reflection. By connecting automated transactions, backend evidence, IT controls, and digital trust, the case provides a practical classroom exercise for undergraduate or graduate Accounting Information Systems, IT Audit, or Audit courses.

Keywords: IT General Controls, Digital Trust, Ticketing Bots, Internal Controls, AI-Assisted Coding, Accounting Information Systems

*

I. THE CASE

Important Disclaimer

This case must be completed only within the sandboxed classroom website provided by the instructor. Students must not use automation techniques on real ticketing websites, commercial platforms, or any system that they do not own or do not have explicit written permission to test.

Unauthorized automation, circumvention of access controls, credential misuse, scraping, or interference with commercial systems may violate website terms of service and applicable civil or criminal laws, including the Computer Fraud and Abuse Act and, in the ticketing context, the BOTS Act.

The purpose of this case is to help students understand IT audit risk, internal controls, transaction processing, and audit evidence in a controlled educational environment. It is not intended to teach or encourage unauthorized automation, ticket scalping, or any activity involving real commercial systems.

SeatGate

SeatGate [1] is a sandboxed online ticketing system used by a university events office to distribute tickets for campus concerts, comedy shows, student-organization events, and selected city events. The system allows students to view upcoming events, select a ticket quantity, and submit a purchase request through a web browser. The school's events office uses SeatGate to manage limited ticket drops for high-demand events and to track completed purchases, remaining inventory, user activity, and exceptions.

The university recently launched a limited online ticket drop for several high-demand events, including a campus concert featuring the university's student-led ballet ensemble. Shortly after the release, students complained that tickets sold out almost immediately. Some students reported that they could not complete a purchase manually before the tickets disappeared. Others claimed that the website appeared to accept purchases faster than an ordinary user could reasonably click through the process. SeatGate's management suspects that automated scripts may have been used by experienced students to purchase tickets faster than ordinary users.

SeatGate was designed to be a controlled transaction system. Each ticket purchase should be initiated by a valid student user, checked against event-level purchase limits, processed against available inventory, recorded in the transaction log, and reflected in management reports. However, management is uncertain whether these controls operated as intended during the ticket drop. In particular, management is concerned about whether the website relied too heavily on front-end restrictions that could be inspected, bypassed, or manipulated by users with basic coding knowledge or access to AI.

Rogers & Rogers LLP[2] will use the COSO Internal Control–Integrated Framework and COBIT as ref-

[1] SeatGate is a fictional classroom system created solely for this educational case. It is not affiliated with, endorsed by or based upon any commercial ticketing company or product.

[2] Rogers & Rogers LLP is a fictional audit firm created solely for this educational case. It is not affiliated with, endorsed by or based upon any commercial audit firm.

erence points for the audit. Under COSO, the audit team will consider whether SeatGate's control environment, risk assessment, control activities, information and communication, and monitoring activities support reliable transaction processing. Under COBIT, the team will consider whether the system's IT processes support governance objectives such as access restriction, processing integrity, logging, exception monitoring, and management review. These frameworks provide a basis for evaluating whether SeatGate's policies are supported by actual system behavior and sufficient audit evidence.

SeatGate's management has engaged Rogers & Rogers LLP to perform an IT audit of the ticketing system. You are a junior IT auditor on the engagement team. Your senior manager, R Rogers, has asked you to evaluate whether SeatGate's controls over online ticket purchases are designed and operating effectively. The engagement focuses on the system's application controls, access controls, logging controls, and monitoring procedures during limited ticket releases.

The events office has the following stated policies:

1. Each student user may purchase no more than one ticket per event.
2. Only valid users are able to complete purchases.
3. Ticket inventory should update immediately after each sale.
4. Automated or suspicious purchasing behavior should be logged, and an exception report should be generated.
5. Every ticket purchased should have a timestamp in the admin view.

Figure 1 summarizes the relationship among the audit standard, the internal-control framework, the IT-control framework, and the SeatGate engagement. PCAOB AS 2201 motivates the audit task. COSO provides the broad internal-control lens. COBIT provides the IT-specific control lens used to evaluate access controls, application controls, logging, exception reporting, and monitoring.

Typically in practice, external IT audits are more concerned with financial-statement or internal controls. The audit team asks whether the information system supports reliable transaction processing, complete records, appropriate access restrictions, and sufficient audit evidence. Conversely, internal audit teams may take a more operational and backend-oriented view. They may examine how the system is configured, how transactions are processed, how logs are generated, how exceptions are reviewed, and how management monitors system performance and risk.

Other than COSO and COBIT, NIST also fits naturally into this assignment as it relates to more technical details of the ticketing system. It is useful for thinking about automated purchasing, suspicious activity, logging, exception reporting, incident response, and system recovery. However, in this case, the main audit work remains framed by AS 2201, COSO, and COBIT because the engagement focuses on whether SeatGate's stated policies are actually enforced by system behavior.

SeatGate's management has asked your audit team to determine whether the ticketing system's controls are operating effectively. As part of your review, you will examine the website's visible structure, observe how users interact with the purchase process, compare front-end behavior with backend records,

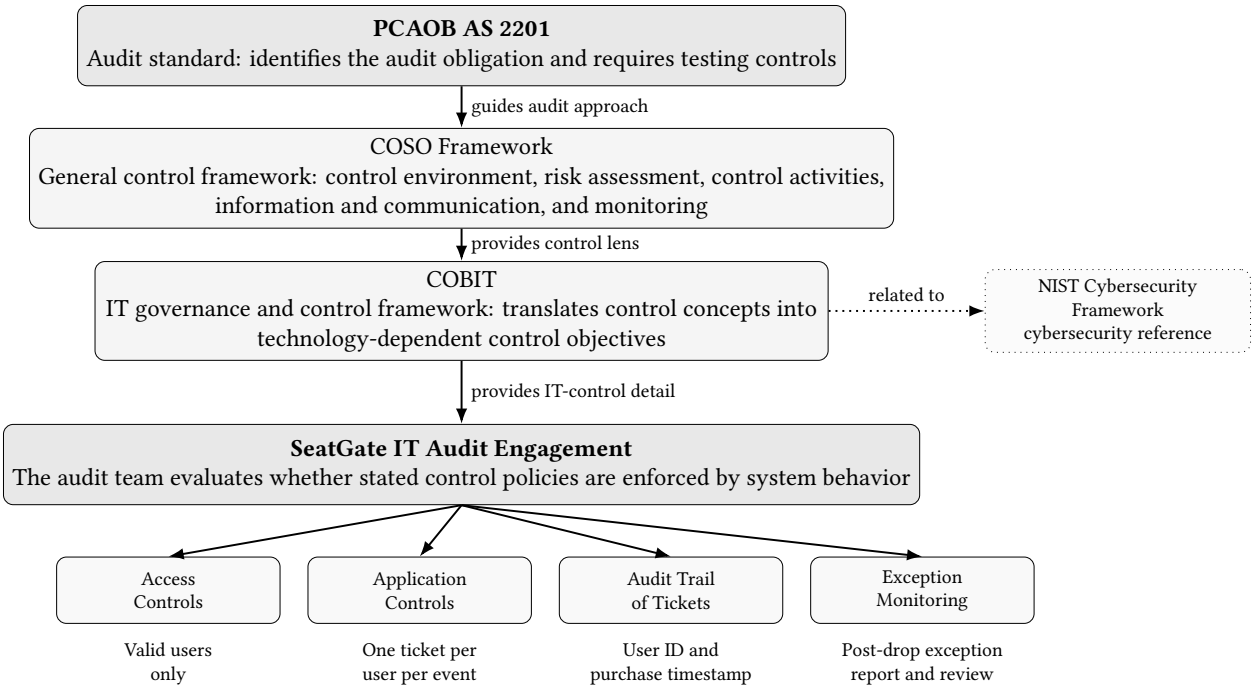

Figure 1: Relationship among PCAOB AS 2201, COSO, COBIT, NIST, and the SeatGate IT audit engagement

and evaluate whether the system provides sufficient evidence for management to detect and respond to suspicious purchasing activity. You will then identify control weaknesses, assess their implications, and recommend improvements consistent with COSO and COBIT control principles.

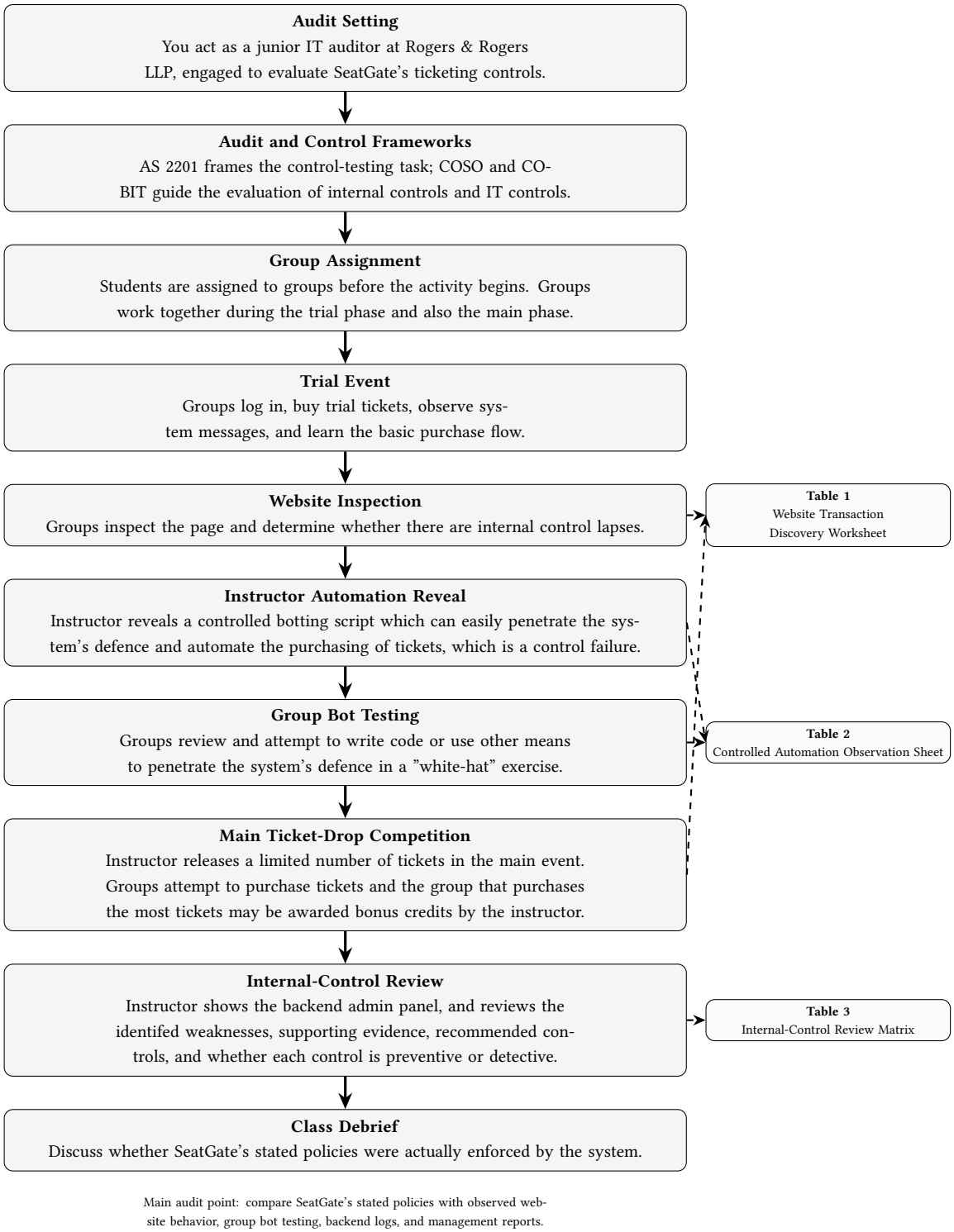

Figure 2: Flow of the SeatGate IT audit case

Figure 2 summarizes the flow of your audit work. You begin as a junior IT auditor evaluating SeatGate's ticketing system. The case is framed by AS 2201, with COSO providing the broad internal-control lens and COBIT providing the IT-control lens. You then move from group assignment to the trial event,

website inspection, controlled automation demonstration, student bot development and trial testing, the main ticket-drop competition, backend evidence review, internal-control review, and class debrief.

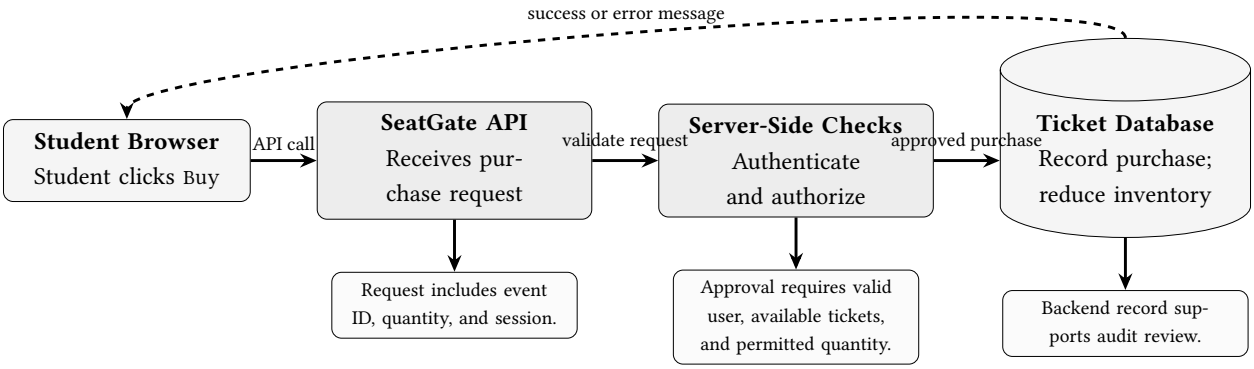

Figure 3: Sandboxed SeatGate purchase request and backend processing flow

Figure 3 shows what happens in the sandboxed SeatGate environment when a student clicks the Buy button. The browser does not directly change the official ticket records. Instead, it sends an API request to the SeatGate ticketing server with information such as the event, requested quantity, and the user's session. The server is responsible for authenticating the user, authorizing the transaction, checking purchase limits and inventory, and deciding whether the request should be approved. If the request is approved, the server updates the database by recording the purchase and reducing the remaining ticket inventory, then sends a success message back to the browser. If the request fails one of the checks, the server returns an error message instead. From an audit perspective, this flow matters because reliable controls must operate on the server and in the database, not only on the visible webpage that users can inspect, refresh, or automate.

Requirements

Before the main ticket drops begin, you will complete a trial event. The trial event is designed to help you become familiar with the SeatGate purchase process. The trial event is not graded unless otherwise stated by your instructor. Log in using the username and password provided by your instructor. Practice viewing the event, submitting a purchase request, and observing the confirmation or error message.

During the trial event and the main ticket drop, inspect the SeatGate website and identify the parts of the webpage that support the purchase process. Document what you observe and consider why each element matters for internal control. After the trial event and instructor automation demonstration, student groups will participate in one main ticket-drop event. Groups will attempt to purchase as many tickets as possible during the limited release. The group that purchases the most tickets may win bonus credits, as determined by the instructor. Your objective is to observe the transaction process, attempt to purchase tickets within the stated rules, and document evidence relevant to the control review.

During the instructor automation reveal, the instructor signs in using the `instructor` account and opens the instructor panel shown in Figure 5. The instructor activates the controlled five-ticket purchase and explains the accompanying pseudocode. The class then reviews the administrator panel in Figure 4

to determine how the automated transaction affected inventory, sales totals, and the instructor's ticket holdings.

Complete Table 1 during or immediately after the trial and main event.

Table 1: Website Transaction Discovery Worksheet

Which event(s) did you buy tickets for?			
Page Area or Function	**What the Page Element Does**	**Possible Internal-Control Issue**	**Audit Relevance**
Home page	Introduces the ticketing website and directs users to available events or login functions	Users may misunderstand where the transaction begins or what actions are available	Helps auditors understand the entry point of the transaction process
Login page	Allows users to enter their email and password to access the system	Weak login controls may allow unauthorized users or bots to access accounts	Relevant to access control and user authentication
User account page	Displays available balance and tickets already purchased	Balance or purchase history may not match backend records	Relevant to access control, authorization, and completeness of transaction records
Cart page	Requires users to input information before they can continue with the purchase process	Required input fields may slow manual users but may not prevent automated requests if the backend does not validate the information	Relevant to application controls, transaction authorization, and whether the control is enforced on the frontend or backend

After reading the table, write a short process narrative:

A user begins by _____. The system then _____. A ticket purchase is completed when _____.

After the trial event, you will observe an instructor-led controlled automation demonstration within the SeatGate sandbox. Compare the automated process with your own manual purchasing experience. Work in your group to observe how there is internal control failure on the ticketing system. Compete with the other teams to see which group purchases the most tickets. After the main ticket drop, you will also review selected backend evidence shown by the instructor that management receives after the ticket drops. This evidence may include transaction logs, failed purchase attempts, exception reports, user activity records, management reports, or other records relevant to the control review.

Students should also review the complete user population displayed in the administrator panel. In particular, they should consider whether every account corresponds to an authorized participant and whether the presence of the `fake-purchase` account indicates a weakness in user provisioning, account review, or management monitoring.

Use the ticket drops to observe the transaction process, identify possible control deficiencies, and complete Table 1, the Website Transaction Discovery Worksheet. In particular, consider whether the system enforces purchase limits, validates users, logs suspicious behavior, updates inventory correctly, and provides management with timestamps and backend evidence for each ticket purchase.

The automation demonstration shows how automated interaction can occur when a website exposes predictable page elements or does not adequately restrict rapid transaction attempts. The purpose of this demonstration is not to teach ticket scalping. The purpose is to show how automated behavior can expose weak digital controls and why IT auditors must understand both front-end activity and backend evidence.

Complete Table 2.

Table 2: Controlled Automation Observation Sheet

Question	Response
What did the script attempt to do?	
Did the system block the automated behavior?	

Using your observations from the ticket drop, website inspection, automation demonstration, and backend evidence, identify control weaknesses and recommend improvements. This requirement should be completed in groups unless otherwise instructed.

Complete Table 3.

Table 3: Group Work: Internal-Control Review

Observed Weakness	**Recommended Control**	**Control Type**

During the debrief, in your groups, be prepared to discuss one observed weakness, the evidence supporting that weakness, and your recommended control. Consider whether a transaction can be recorded by the system but still fail to satisfy the underlying control objective. You should also consider whether SeatGate's controls enforced purchase limits, validated users, updated inventory correctly, logged suspicious behavior, and produced exception reports that management could review.

As part of the class discussion, address the following questions:

1. What was the most important internal-control weakness revealed by the ticket-drop exercise?
2. What evidence would an IT auditor need to determine whether management's stated policies were actually enforced?

Any code should be used only within the SeatGate sandbox and only for activities authorized by your instructor. Do not apply automation techniques to real ticketing websites or external systems. The purpose of the exercise is not to obtain tickets using automated scripting, but to observe how IT auditors need to understand the backend process.

All materials, including the video tutorials, the worksheets with suggested answer keys can be found at https://github.com/hunternbh/acc3202-ticketing-bot.

Motivation for the Case

The Sarbanes-Oxley Act of 2002 (SOX) requires management to maintain and assess internal controls over financial reporting. Under PCAOB AS 2201, auditors evaluate these controls using a top-down approach that begins with financial statement risks and then focuses on the controls most relevant to material misstatement. Information Technology (IT) general controls are central to this process because they support the reliability of application controls, transaction processing, and system-generated data.

In a financial-statement audit, the IT audit team is generally not asked to evaluate every operational aspect of the system. The team usually focuses on IT controls that affect financial reporting, including controls over access, program changes, application processing, interfaces, system-generated reports, and the completeness and accuracy of transaction data. When relevant to the audit, the IT audit team may inspect backend code, configuration settings, database rules, or system logic to determine whether the implemented control matches the stated control requirement. The audit concern is not simply whether a system function exists, but whether the function is designed and implemented in a way that supports reliable financial reporting.

IT auditors examine whether information systems support reliable transaction processing and system-generated audit evidence (Green, Box, and Okas, 2026). In a financial-statement audit, this review is usually scoped to controls that affect financial reporting. These may include user access, change management, application processing, system-generated reports, interfaces, and audit trails (Lee and Sawyer, 2019). The audit team may also inspect backend code or configuration logic when necessary to determine whether the system enforces the control requirement as documented. For example, if management states that each user may purchase only one ticket for an event, the relevant audit question is whether the backend actually checks prior purchases before approving another transaction.

While these controls are a basic component of IT audit, contemporary IT auditors must also understand automated transactions, platform governance, data analytics, cybersecurity controls, artificial intelligence, and the risks created by rapidly changing digital systems (Dzuranin and Mălăescu, 2016; Li and Goel, 2025). These topics matter because modern transaction systems are not only accounting systems in the narrow sense. They are digital platforms in which user access, application logic, backend processing, monitoring, and system-generated reports jointly determine whether recorded transactions are complete, accurate, authorized, and reviewable.

IT audit procedures often include walkthroughs, inspection of system logic, and testing scenarios designed to evaluate whether the system processes transactions as intended. Auditors may create normal and exception scenarios, process alternative transaction paths, and compare the expected outcome with the system-recorded outcome. In the SeatGate setting, this means testing whether a valid user can purchase one ticket, whether a second purchase is rejected, whether inventory is reduced correctly, whether failed or suspicious attempts are logged, and whether management reports reflect the underlying transaction activity.

Technical fluency is therefore becoming part of audit competence. Coding is increasingly relevant to accountants because it allows professionals to automate repetitive tasks, inspect data and systems directly, and understand the logic embedded in digital business processes (ACCA, 2021). Recent professional

commentary similarly notes that accountants are already using AI for automation, data analysis, technical research, and document preparation (Kenney, 2025). These developments suggest that auditors need a certain level of coding and AI literacy to evaluate system behavior, validate automated outputs, and communicate effectively with technical specialists.

A challenge for AIS instructors is to teach these technical skills without turning the course into a computer science course. The instructional setting must be simple enough for students to understand quickly, but rich enough to show how system design affects internal controls. We use online ticketing to illustrate the process. Students understand the business process, the scarcity of tickets creates an intuitive incentive for automation, and the transaction flow allows instructors to connect technical behavior to audit concepts. The *Wall Street Journal* has reported on ticket scalping, resale practices, and ticketing fees, reflecting a broader phenomenon that students are likely to recognize (Steele, 2025; Fanelli and Michaels, 2026). Using a real-life example gives the case verisimilitude and engages students, while allowing instructors to keep the implementation focused on internal controls, digital transaction processes, and basic coding.

We refer to a well publicized antitrust case, United States v. Live Nation Entertainment, filed in 2024. The Federal Trade Commission, joined by seven states, sued Live Nation and Ticketmaster for allegedly engaging in illegal ticket resale practices and deceiving artists and consumers about prices and ticket limits. The FTC alleged that Ticketmaster publicly claimed to protect fans from brokers while profiting from brokers who used thousands of accounts and proxy IP addresses to bypass stated ticket limits, acquire large volumes of tickets, and re-sell them on Ticketmaster's own resale platform at substantial markups (Federal Trade Commission, 2025).A settlement was proposed in March 2026, including several limitations on Ticketmaster in the sale of tickets.

From an IT audit perspective, the allegations raise direct control questions. For a financial-statement audit, the relevant questions concern whether the system-generated transaction data are complete, accurate, authorized, and supported by effective IT controls. Were purchase limits actually enforced by backend logic? Did the system record each approved and rejected transaction with sufficient detail? Could management rely on the system reports used to evaluate sales, refunds, fees, or other financial activity? Broader questions about platform fairness, broker relationships, resale strategy, and day-to-day operational monitoring may also matter, but those issues are more naturally within the scope of management review, compliance, or the firm's internal audit function. The classroom case therefore uses operationally familiar ticketing activity to teach the financial-reporting and control-testing logic that IT auditors need to understand.

Hence, we present a case that uses a sandboxed classroom ticketing website to teach IT audit and internal-control concepts through a controlled ticket-drop exercise. Students first participate in three limited ticket releases as ordinary users. They then inspect the website's HTML structure and API calls, use AI-assisted "vibe coding" to understand automated interaction, and evaluate whether the backend implementation is consistent with the system's stated control requirements. The purpose of the case is to show how digital transactions are initiated, authorized, processed, logged, monitored, and reported, and how auditors can compare stated policies with implemented system logic.

After the exercise, the instructor reveals backend logs and the automation script, allowing students to compare front-end behavior, backend code logic, and system-recorded activity. Students then identify control weaknesses, evaluate audit evidence, and recommend preventive and detective controls. The emphasis is not on operational policing of the ticketing platform, but on understanding whether the system's implemented controls support reliable transaction processing and sufficient audit evidence.

The case uses gamification (Blix, Edmonds, Edmonds, and Sorensen, 2026) and a real-world ticket-scalping context (Gantman, 2024) to connect coding, transaction processing, backend logging, and internal-control reasoning. By requiring students to inspect the DOM, observe automated transactions, analyze exception reports, and prepare audit documentation, the case responds to calls for AIS education to integrate technical fluency, AI-related skills, and audit judgment (AICPA, 2025; Dzuranin and Mălăescu, 2016; Li and Goel, 2025; ACCA, 2021; Kenney, 2025).

Past Literature on IT Controls

Following the structure of Lee and Sawyer (2019), we review recent instructional cases, in the last five years, related to internal controls and IT controls from three leading accounting education journals: *Issues in Accounting Education*, *Journal of Accounting Education*, and *AIS Educator Journal*. Table 4 summarizes the cases most closely related to this study.

A distinctive feature of this case, different from past literature, is that the instructor controls both the front end and the back end of the SeatGate system. Students therefore do not only review a written case, a diagram, or a static dataset. They audit a live system in which website behavior, user actions, ticket purchases, inventory updates, backend logs, and exception reports can be compared. This design allows students to evaluate whether stated control policies are actually enforced by the system. The case is also practitioner-oriented. Students practice activities that resemble practice, including inspecting transaction flows, tracing system evidence, evaluating control design and operation, identifying weaknesses, and recommending improvements.

Table 4: Recent instructional cases on internal controls and IT controls (2021–2026)

Year	Citation	Full title	Primary topics & software	Main task related to IT control
2021	Lawson and Street (2021)	Detecting dirty data using SQL: Rigorous house insurance case	Data quality, claims analytics, preventative IS controls; software: SQL	Query claims data, identify integrity exceptions, and recommend preventative information-system controls.
2022	Boss et al. (2022)	Accountants, Cybersecurity Isn't Just for "Techies": Incorporating Cybersecurity into the Accounting Curriculum	Cybersecurity, disclosure, audit/tax risks, short cases; artefacts: case materials and 10-K-style disclosures	Analyse six cyber scenarios and evaluate risk, control, reporting, and assurance implications.
2022	Brown and Lightle (2022)	Only Reliable Parts and Supplies, Inc.: Assessing and documenting the design of internal controls	Sales-process controls, flowcharting, audit documentation; software: flowcharting/word-processing tools	Flowchart the sales cycle, assess control design, and draft a memo on internal-control weaknesses.
2023	Parlier and Lee (2023)	Inventory analytics: A teaching case using excel and Alteryx	Audit analytics, inventory anomalies, exception analysis; software: Excel and Alteryx	Use analytics to investigate inventory anomalies and discuss implications for control monitoring and audit evidence.
2023	Hawk (2023)	Building Higher-Order Thinking Skills Through Assessment of User Account Access	IT general controls, user access management, audit analytics; software: Excel and CaseWare IDEA	Test user-access controls, analyse exceptions, and conclude on ITGC effectiveness.
2024	Boss et al. (2024)	Be an Expert: A Critical Thinking Approach to Responding to High-Profile Cybersecurity Breaches	Cyber breaches, internal audit, incident response; software: case materials	Evaluate breach causes, prevention, and organisational response from security, CIO, and internal-audit perspectives.
2024	Soonawalla and Wakefield (2024)	Stopping Hamburglars: Applying Effective Internal Control	COSO, fraud triangle, business-process controls; software: none stated	Diagnose control weaknesses and redesign a stronger internal-control system for a fast-food operation.
2024	Zeller et al. (2024)	Workbook design and controls: A framework	Spreadsheet controls, workbook design, reviewability; software: Excel	Redesign a workbook using control principles that improve reliability, transparency, and review efficiency.
2024	Mascha and Miller (2024)	Applying Internal Control Concepts Using Database Design: An Educational Case	Internal controls, transaction processing, database design; software: Microsoft Access	Build database tables, forms, queries, and reports that embed internal-control concepts in system design.
2024	Jiang et al. (2024)	The Sunday Standstill: An Accounting Information System Upgrade Role-Play	AIS implementation, change management, information security policy, internal controls; software: role-play case	Evaluate upgrade decisions and control consequences during an AIS change scenario.
2025	Bloch et al. (2025)	The Falling House of Wirecard	Fraud, governance, internal-control failure in digital payments; software: none stated	Analyse red flags, governance breakdowns, and internal-control weaknesses behind a digital-platform collapse.
2026	Janvrin et al. (2026)	Colonial Pipeline: Responding to a Ransomware Attack in the Energy Sector	Ransomware, cyber governance, contingency planning, resilience; software: none stated	Assess incident response, cyber-control weaknesses, and resilience planning after a major ransomware attack.

Note: Entries are limited to studies from *Issues in Accounting Education*, *Journal of Accounting Education*, and *AIS Educator Journal* (2021–2026) whose public publisher metadata put them as teaching cases, educational cases, case reports, role-plays, or paired teaching-note items.

Learning Objectives

After completing the case, students should be able to:

1. Identify the main steps in a digital ticket-purchase transaction.
2. Explain how automated purchasing can create internal-control risks.
3. Distinguish between a transaction that is recorded and a transaction that is properly authorized.
4. Evaluate whether purchase-limit, authorization, logging, monitoring, and exception-reporting controls are operating effectively.
5. Recommend preventive and detective controls to reduce bot-like purchasing behavior.
6. Prepare audit documentation that connects observed system behavior to internal-control findings.

Instructor Implementation Guidance

Several materials accompany this case, including an instructional video, a PowerPoint deck for the student debrief, and student worksheets. These materials are available in the case GitHub repository at https://github.com/hunternbh/acc3202-ticketing-bot. The instructional video is also available at https://youtu.be/gP3dJlWociM.

SeatGate is designed as a sandboxed IT audit case in which students act as junior IT auditors evaluating whether the system's stated control policies are actually enforced by system behavior and supported by audit evidence. The case connects AS 2201, COSO, and COBIT to a live digital transaction process involving authentication, ticket inventory, purchase authorization, logging, exception monitoring, and backend evidence. The accompanying materials are provided to help instructors implement the exercise, guide student work, and debrief the case after the ticket-drop activity.

Instructors should clarify that the case uses a simplified setting to distinguish external IT audit work from broader operational review. In a financial-statement audit, the IT audit team generally focuses on controls that affect the reliability of financial reporting and system-generated evidence. The team may inspect backend code, database rules, or configuration settings to determine whether the implemented logic matches the documented control requirement. Internal audit, by contrast, is more likely to examine the broader operational aspects of the ticketing process, such as customer experience, platform fairness, business policy compliance, and ongoing operational monitoring.

Before the activity begins, the instructor should seed the database. Seeding refreshes the exercise data, resets previous purchases, recreates the user accounts, and creates the trial event. Students should use the trial event to learn the transaction flow, inspect the website, and test any instructor-approved AI-assisted code.

After the trial event, the instructor will reveal a controlled automation script within the SeatGate sandbox. Student groups will then use the trial event to understand, adapt, and test instructor-approved botting code before participating in one main ticket-drop event. To generate student interest, instructors

may decide to reward the group with the highest number of tickets bonus credits. whether ticket purchases count toward the course or case grade.

The instructor should release in one shot for the main event using the provided backend commands, depending on the number of students. Releasing a controlled number of tickets gives the class excitement as students work in groups to buy as many tickets as they can for their groups. For example, if there are N students, each event may have approximately $2N\%$ tickets available.

Students should use AI-assisted coding to understand how the website works, automate parts of the purchase process in the sandboxed environment, or test whether SeatGate's controls prevent or detect rapid purchase attempts. Instructors should clearly state that this activity is limited to the SeatGate sandbox. The purpose of the exercise is not to teach ticket scalping or circumvention of real ticketing systems. The purpose is to show how automated behavior can expose weak digital controls and why IT auditors must understand both front-end behavior and backend evidence.

After the ticket drops, the instructor should reveal selected backend reports. These may include transaction logs, failed purchase attempts, exception reports, user activity records, ticket ownership records, wallet balances, or other evidence relevant to the control review. The instructor may also demonstrate a controlled automation script. The demonstration should help students compare manual purchasing behavior with automated purchasing behavior. Students should observe whether the system blocks rapid activity, logs suspicious behavior, enforces purchase limits, updates inventory correctly, and produces evidence that management could review.

Table 6 provides one possible implementation structure.

Student performance can be assessed using both technical and audit-based deliverables. The ticket-drop result provides evidence that students engaged with the website and understood how digital transactions can be affected by timing, automation, and control design. However, the main grading emphasis should be placed on the quality of students' audit reasoning. Instructors may choose to make the ticket-purchase outcome a small portion of the grade or treat it as participation only. This reduces the risk that students focus on "winning" the ticket drop rather than understanding the control implications.

The case may be assessed using two sources of evidence: a required case worksheet and a post-case perception survey. The worksheet asks students to document their observations from the ticketing website, the ticket-purchase process, the automation demonstration, and the backend evidence provided by the instructor. These responses allow the instructor to assess whether students can connect technical observations, such as rapid purchasing, exposed page elements, backend validation logic, processing alternatives, purchase-limit behavior, and backend logs, to audit concepts such as authorization, transaction validity, monitoring, exception reporting, and server-side validation. The post-case perception survey asks whether the exercise improved students' understanding of IT audit, internal controls, automated purchasing risks, AI-assisted coding, and the importance of technical fluency for auditors.

The instructor debrief should emphasize that a transaction can be recorded by the system but still fail to satisfy the underlying control objective. Front-end restrictions are not sufficient if important controls are not enforced on the server side. A valid audit trail should include enough information to evaluate user identity, timing, authorization, inventory effects, and exception behavior. Automated purchasing risk is

Table 6: Implementation Guidance

Phase	Description	Who Performs Phase	Estimated Time
Phase 1	Introduce the SeatGate audit setting, sandbox limitation, stated control policies, group structure, and the AS 2201, COSO, and COBIT framing. Students are assigned to groups before the activity begins.	Instructor	15–20 minutes
Phase 2	Students complete the trial event, inspect the website transaction process, and begin the Website Transaction Discovery Worksheet. During the trial, students explore the system and practice the basic purchase flow.	Students/groups	05–10 minutes
Phase 3	Instructor reveals a controlled automation script that can purchase tickets within the SeatGate sandbox. Students compare the automated process with the manual purchase process.	Instructor/class	10–20 minutes
Phase 4	Student groups use the trial event to figure out how to write, adapt, and test botting code within the sandbox.	Students/groups	10–20 minutes
Phase 5	Instructor opens one main ticket-drop event. Student groups attempt to purchase tickets during the limited release. The group that purchases the most tickets wins bonus credits.	Students/groups	10–20 minutes
Phase 6	Instructor reveals backend evidence, including backend logic, logs, failed attempts, processing alternatives, ticket ownership records, and reports relevant to control testing.	Instructor/class	10–20 minutes
Phase 7	Students complete the internal-control review in groups and recommend preventive or detective controls.	Students/groups	10–15 minutes
Phase 8	Instructor leads the class debrief and connects observations to audit evidence, authorization, transaction validity, logging, and monitoring.	Instructor/class	10–15 minutes

Table 7: Suggested Grading Allocation

Component	Suggested Weight
Participation in ticket drops and sandbox activity	0-10%
Website Transaction Discovery Worksheet	30%
Controlled Automation Observation Sheet	20-30%
Internal-Control Review	30%

not only a technical issue; it is also an internal-control, monitoring, and governance issue. COSO provides the broad internal-control lens, while COBIT helps translate that lens into IT-control objectives such as access restriction, processing integrity, logging, exception monitoring, and management review.

The SeatGate environment is a sandboxed classroom system. Instructors should remind students that automation should not be used on real ticketing websites or external systems. The case should be framed as an IT audit and internal-control exercise, not as an exercise in ticket scalping. The appropriate learning objective is to understand how digital transactions are initiated, authorized, processed, logged, monitored, and reported.

After the debrief, instructors may use the following suggested answer key to guide discussion and grading. The examples are not exhaustive; students may identify additional weaknesses or propose alternative controls. Strong answers should connect the observed system behavior to a specific control objective, explain why the weakness matters, and recommend a control that would either prevent the issue or help management detect and review it.

Table 8: Suggested Answer Key: Internal-Control Review

Framework	Observed Weakness	Recommended Control	Control Type
COBIT	Users can buy more than one ticket, even if the stated policy is one ticket per user or one ticket per event.	Add a backend validation rule that checks whether the user has already purchased a ticket for the event before approving another purchase.	Preventive application control
COBIT	The system does not produce clear exception reports for rapid repeated requests	Create exception reports for abnormal activity, such as multiple purchase attempts by the same user, repeated failed purchases, unusually fast requests, or purchases from the same IP address.	Detective control
COBIT	The system does not clearly monitor whether purchase requests are submitted manually through the website or automatically through a script.	Use rate limits, request throttling, bot detection, and monitoring rules to identify repeated requests within a short time window.	Preventive and detective control
COBIT	The class and date control requirement appear on the website interface but are not fully enforced by the backend.	Move critical validation to the backend. The frontend can guide users, but the backend must enforce access control, purchase limits, inventory checks, and transaction approval.	Preventive application control
ASC	Fake account initiated in the database with $20	Check integrity of revenue and generate audit report and manual review	Detective control
ASC	Revenue not calculated properly due to the fake account	Check integrity of revenue and generate audit report and manual review	Detective control
NIST	The purchase API is easily discoverable through the browser's Network tab. Students can identify the endpoint, request method, and request payload used to buy tickets.	Do not rely on the frontend interface as the main control. The backend should validate every purchase request, verify user authorization, enforce purchase limits, check inventory, and reject abnormal or repeated requests.	Preventive and detective application control
NIST	There is no CAPTCHA, bot challenge, or other friction before completing a purchase.	Add bot-detection controls such as CAPTCHA, rate limiting, request throttling, or behavioral monitoring for rapid repeated requests.	Preventive control; detective if suspicious activity is logged and reviewed

Classroom User Roles and Embedded Audit Evidence

The SeatGate classroom environment uses several predefined user roles. Each student receives an anonymous username, such as `student01`, and a password from the instructor. These accounts allow students to

participate in the trial and main ticket releases without placing real student names in the public repository.

The system also contains a deliberately created user called `fake-purchase`. This account represents an embedded control issue in the case. It appears in the system alongside the ordinary users and may hold tickets even though it does not correspond to an actual student participating in the exercise. The account is intended to prompt students to consider whether management adequately reviews the user population, identifies unauthorized or fictitious accounts, and reconciles ticket ownership with approved users. Students are not told the significance of this account before the exercise because identifying it forms part of the audit investigation.

Two privileged accounts support the classroom exercise. A user who signs in with the `admin` account can access the administrator panel shown in Figure 4. The panel displays the total number of users, tickets sold, revenue, trial and main ticket availability, ticket inventory, and the ticket holdings of all users. It also permits the administrator to reset the classroom database and release additional tickets. The panel therefore functions both as an operational interface and as a source of system-generated audit evidence.

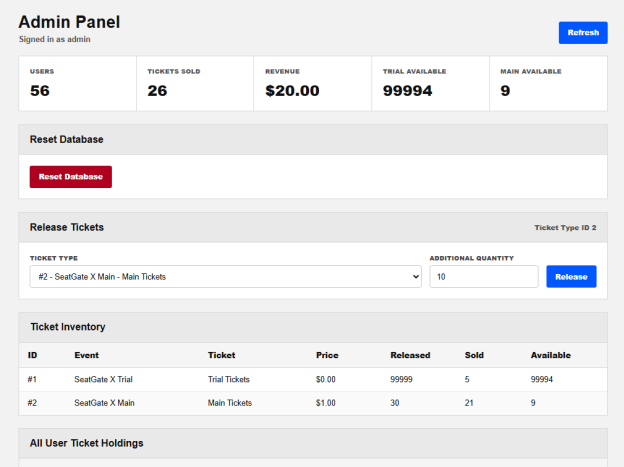

Figure 4: SeatGate administrator panel accessible through the `admin` account

A user who signs in with the `admin` account can also access the instructor demonstration panel shown in Figure 5. This panel contains an **Automated Buy** button and pseudocode describing the automated purchase sequence. The demonstration authenticates the instructor, retrieves the available ticket type, and sends a purchase request for five tickets in one transaction.

The instructor panel demonstrates that restrictions presented through the ordinary student interface may not be sufficient if equivalent restrictions are not enforced by the backend. After the automated purchase, students can compare the instructor's ticket holdings, the number of tickets sold, and the remaining inventory in the administrator panel. This comparison allows students to examine whether the transaction was authorized, whether inventory was updated correctly, whether the unusual quantity was detected, and whether management received sufficient evidence to investigate the activity.

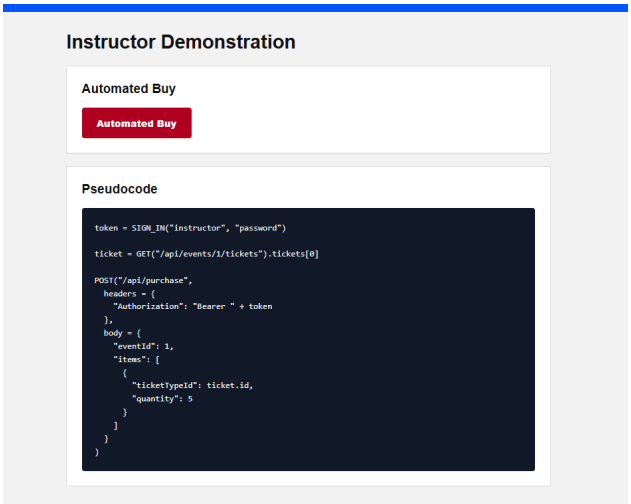

Figure 5: SeatGate instructor panel displaying the controlled automated-purchase demonstration and its pseudocode

Case Efficacy

The case was tested with undergraduate Bachelor of Accountancy (BAcc) students at a large public college in the northeastern United States. We evaluated case efficacy using three sources of evidence: a short pre- and post-case assessment, a post-case student perception survey, and practitioner review. This approach is consistent with prior AIS case research that evaluates student learning and student perceptions of case usefulness (Lee and Sawyer, 2019).

The pre- and post-case assessment was designed to measure students' understanding of backend validation, transaction authorization, transaction logs, automated purchasing risk, exception reporting, preventive and detective controls, browser-based transaction inspection, and the role of COSO and COBIT in evaluating IT controls. Students completed the pre-assessment before beginning the case and completed the post-assessment after the SeatGate exercise. The assessment used the same ten items at both points in time; item wording and question-level results appear in Table 11.

The second source of evidence was a post-case perception survey. Students indicated the extent to which they agreed with statements about the case using a five-point scale from "strongly disagree" to "strongly agree." The survey items addressed perceived learning about IT audit, automated purchasing risk, AI-assisted coding, internal-control review, and overall case usefulness. The complete item wording and question-level response scores appear in Table 12.

The survey also included the two open-ended questions shown in Table 9.

Table 9: Open-Ended Student Perception Survey Questions

Item	Question
1	What was the most useful part of the case?
2	What change would improve the case for future students?

Together, these three sources of evidence provide a concise assessment of case efficacy. The pre/post assessment measures conceptual understanding before and after the activity. The post-case survey captures students' perceptions of learning, realism, and engagement. The practitioner review provides evidence that the exercise reflects the logic of real IT audit work while remaining appropriate for entry-level students.

Results

The analysis matched forms using the anonymous student-generated codes written on the top of each two-page instrument. In total, we collected 56 pre-assessment forms, 38 post-assessment forms, and 35 post-case perception surveys. Twenty-five pre/post assessment pairs were matched using exact anonymous codes. Unmatched forms were retained in the response tallies but excluded from paired score comparisons. Blank responses were treated as incorrect when calculating assessment scores and as missing when calculating survey means.

Table 11 follows the question-level reporting format used in the reference case and presents the mean response score for every assessment item. Each item was scored 0 for an incorrect or blank response and 1 for a correct response. Because the returned instruments were unbalanced and many two-page assessment forms included unanswered items on the second page, the item-level results are more informative than an aggregate score. The results show encouraging gains on several targeted IT audit concepts. Scores increased on the importance of transaction logs, the need to ask whether a recorded transaction was authorized under stated control rules, automated purchasing risk, backend enforcement of purchase limits, and exception-reporting evidence. These gains align with the case's core learning objectives and suggest that the exercise helped students connect technical system behavior to audit evidence and internal-control evaluation.

Table 11: Pre- and Post-Assessment Questions and Mean Item Scores

Question	Pre M ($n = 25$)	Post M ($n = 25$)
1. Which statement best describes the role of backend validation in the SeatGate ticketing system?	0.84	0.80
2. SeatGate states that each user may purchase only one ticket per event. Which control would best enforce this policy?	0.84	0.76
3. Why is a transaction log important to an IT auditor reviewing the ticket drop?	0.76	0.80
4. A purchase appears in the system records. What additional question should an auditor ask before concluding that the transaction was valid?	0.44	0.48
5. Which observation most directly suggests a risk from automated purchasing?	0.36	0.56

Table 11 continued

Question	Pre M ($n = 25$)	Post M ($n = 25$)
6. Which item would be most useful in an exception report for detecting bot-like behavior?	0.44	0.44
7. What is the main weakness when an important purchase limit is shown on the webpage but not enforced by the backend?	0.36	0.52
8. Which statement best distinguishes preventive and detective controls in the SeatGate case?	0.40	0.28
9. Why might students inspect the browser Network tab during the case?	0.28	0.20
10. Which statement best explains why COSO and COBIT are both useful in this case?	0.44	0.20

Notes: M is the mean item score. Each item was scored 0 (incorrect or blank) or 1 (correct). Sample sizes include only students with exact code-matched pre- and post-assessments.

The perception survey results in Table 12 were generally favorable. Consistent with the reference case, the table reports the wording, item-specific sample size, and mean response score for every closed-ended question. Mean ratings exceeded the neutral midpoint of 3.0 for every item. The highest-rated items were the risk of relying on AI-generated code without validation ($M = 3.71$), the usefulness of the internal-control review in connecting technical system behavior to audit findings ($M = 3.69$), and the recommendation to continue using the case with accounting students ($M = 3.66$). The overall mean across the ten survey items was 3.54.

Table 12: Post-Case Student Perception Survey Questions and Response Scores

Question	n	M
1. As a result of the case, I have a better understanding of how to test IT controls in a digital transaction system.	35	3.51
2. As a result of the case, I better understand how automated purchasing can create internal-control risks.	35	3.60
3. As a result of the case, I better understand the difference between a recorded transaction and an authorized transaction.	35	3.57
4. As a result of the case, I improved my ability to inspect basic HTML or webpage elements.	35	3.34
5. As a result of the case, I better understand how AI-assisted coding can help auditors examine digital systems.	35	3.57
6. As a result of the case, I better understand the risks of relying on AI-generated code without validation.	35	3.71
7. The internal-control review helped me connect technical system behavior to audit findings.	35	3.69
8. The case was a positive learning experience.	35	3.40
9. The case was an interesting assignment.	35	3.34
10. I recommend continuing to use this case with accounting students.	35	3.66

Notes: Responses were scored from 1 (strongly disagree) to 5 (strongly agree). M is the mean response score. The reported n includes all usable post-case perception survey responses.

The open-ended responses were consistent with the quantitative ratings. Students most often described the useful parts of the case as learning about internal controls, observing ticket-count evidence, using trial runs to understand the system and automation process, inspecting backend behavior, and experimenting with coding approaches. Suggested improvements centered on clearer setup guidance, more organization, additional practice opportunities, and more introductory support for students who were less familiar with browser tools, HTML, or using the Inspect feature on different operating systems.

Practitioner Feedback

To assess the case's efficacy, we also consulted a Big 4 IT auditor with five years of experience who is an author on the team. She reviewed the SeatGate exercise and confirmed that the exercise was identical to what happens in real IT audit cases in its core audit steps: auditors obtain an understanding of a transaction process, identify where control failures could occur, inspect system evidence, and evaluate whether implemented controls operate as stated. Based on this review, we retained the core IT audit structure and tweaked selected details to make the case more entry-level for students, including the sandboxed environment, the limited automation demonstration, and the focus on introductory control concepts.

Teaching Notes

Teaching notes are included in the Appendix. The teaching notes provide instructor setup instructions, suggested classroom commands, guidance for releasing tickets, sample internal-control weaknesses, and suggested debriefing points. Instructors should not distribute the teaching notes to students before the exercise.

References

ACCA (2021). Coding: as a professional accountant, why you should be interested. Technical report.

AICPA (2025). AICPA and CIMA Launch AI Accelerator Skills Program To Prepare Finance Leaders for an AI Enabled Future.

Blix, L., C. Edmonds, J. Edmonds, and K. Sorensen (2026). Social Gamification in Online Accounting Courses and the Impact on Course Engagement. *Issues in Accounting Education 41*(2), 1–27.

Bloch, R. I., K. E. Hunter, and G. Kleinman (2025). The falling house of wirecard. *Issues in Accounting Education 40*(2), 121–132.

Boss, S. R., J. Gray, and D. J. Janvrin (2022). Accountants, cybersecurity isn't just for "techies": Incorporating cybersecurity into the accounting curriculum. *Issues in Accounting Education 37*(3), 73–89.

Boss, S. R., J. Gray, and D. J. Janvrin (2024). Be an expert: A critical thinking approach to responding to high-profile cybersecurity breaches. *Issues in Accounting Education 39*(1), 93–121.

Brown, K. F. and S. Lightle (2022). Only reliable parts and supplies, inc.: Assessing and documenting the design of internal controls. *Journal of Accounting Education 60*, 100783.

Dzuranin, A. C. and I. Mălăescu (2016). The current state and future direction of IT audit: Challenges and opportunities. *Journal of Information Systems 30*(1), 7–20.

Fanelli, J. and D. Michaels (2026, March). Live Nation CEO Grilled Over Ticket Fees at Antitrust Trial. *Wall Street Journal*.

Federal Trade Commission (2025, September). FTC Sues Live Nation and Ticketmaster for Engaging in

Illegal Ticket Resale Tactics and Deceiving Artists and Consumers about Price and Ticket Limits. Last Accessed May 4, 2026.

Gantman, S. (2024). Does Using Real Organizations for Team Projects in an AIS Class Make a Difference? *AIS Educator Journal 19*(1), 17–34.

Green, S., D. Box, and J. Okas (2026). IT Audit Services. https://www.ey.com/en_us/services/technology-risk/information-technology-audit-services.

Hawk, H. (2023). Building higher-order thinking skills through assessment of user account access. *AIS Educator Journal 18*(1), 39–49.

Janvrin, D. J., S. R. Boss, and J. Gray (2026). Colonial pipeline: Responding to a ransomware attack in the energy sector. *Issues in Accounting Education 41*(2), 131–145.

Jiang, R., M. Luttenton-Knoll, P. Hillman, and D. A. Pellathy (2024). The sunday standstill: An accounting information system upgrade role-play. *AIS Educator Journal 19*(1), 50–69.

Kenney, B. A. (2025, June). Real-life ways accountants are using AI.

Lawson, J. G. and D. A. Street (2021). Detecting dirty data using SQL: Rigorous house insurance case. *Journal of Accounting Education 55*, 100714.

Lee, L. and R. Sawyer (2019). IT general controls testing: Assessing the effectiveness of user access management. *AIS Educator Journal 14*(1), 15–34.

Li, Y. and S. Goel (2025). Bridging IT auditors and AI auditing: Understanding pathways to effective IT audits of AI-driven processes. *Advances in Accounting 69*, 100842.

Mascha, M. F. and C. L. Miller (2024). Applying internal control concepts using database design: An educational case. *AIS Educator Journal 19*(1), 1–16.

Parlier, J. and L. Lee (2023). Inventory analytics: A teaching case using excel and alteryx. *Journal of Accounting Education 63*, 100848.

Soonawalla, K. and J. Wakefield (2024). Stopping hamburglars: Applying effective internal control. *Issues in Accounting Education 39*(4), 165–181.

Steele, A. (2025, March). Trump Signs Order Targeting Ticket Scalpers and Fees. *Wall Street Journal*.

Zeller, T., E. Dingrando, and D. Booker (2024). Workbook design and controls: A framework. *Journal of Accounting Education 69*, 100933.

Appendix

A Instruction Manual

This case is accompanied by three instructional videos, student materials, and a public GitHub repository.

Instructional Videos

- **Setup Instructional Video:** https://youtu.be/wHpjUpBzZMc
- **Classroom Briefing Video:** https://youtu.be/hIh2D8scudc
- **Observing Network Flow Video:** https://youtu.be/u1Z670PWvwY

Case Materials

The materials folder contains the student worksheet, student case deck, account materials, automated bot program, consent form, and manuscript PDF. These materials are available at:

https://github.com/hunternbh/acc3202-ticketing-bot/tree/main/materials

The complete public GitHub repository, including the source code and supporting materials, is available at:

https://github.com/hunternbh/acc3202-ticketing-bot

The following manual summarizes the setup process demonstrated in the Setup Instructional Video.

A.1 GitHub Setup

Instructors should first create a GitHub account if they do not already have one. Using a Google account may simplify the registration and connection process.

After signing in, go to:

https://github.com/hunternbh/acc3202-ticketing-bot

Click **Fork** to copy the repository into your own GitHub account. Rename the forked repository to something memorable. The instructional video uses:

`ticketing-bot`

Record the repository name because it must be entered in the frontend configuration later.

A.2 Render Database Setup

Go to:

https://render.com

Create an account and a new Render project. Within the project, create a PostgreSQL database. The database stores the users, ticket inventory, and ticket-purchase activity.

Use any database name, although `ticketing-bot` is recommended for consistency. Select the Free plan.

The free database may expire or wind down after approximately one month. Instructors should therefore avoid creating it too far in advance of the class. If it expires, a replacement database can be created.

A.3 Render Web Service Setup

While the database is being created, add a new Web Service to the same Render project. Connect the Web Service to the GitHub repository forked earlier.

Use any service name, although `ticketing-bot` is recommended. Configure the service as follows:

Table 13: Render Web Service Settings

Setting	Value
Root directory	`server`
Build command	`npm install`
Start command	`npm start`
Instance type	Free

Click **Deploy** to create the Web Service.

When the PostgreSQL database is ready, open the database page and copy its **Internal Database URL**. Return to the Web Service, open the **Environment** settings, and create the following environment variable:

```
DATABASE_URL
```

Paste the Internal Database URL as its value and save the changes. No `SEED_SECRET` variable is required.

A.4 Frontend and Server Configuration

Return to the forked GitHub repository and open:

```
vite.config.ts
```

Edit the `base` value so that it exactly matches the name of the forked repository. For example, if the repository is named `ticketing-bot`, use that name as the base path. Commit the change.

Next, return to the Render Web Service and copy its public domain. Do not include punctuation that is not part of the domain.

In GitHub, open:

```
src/.env.production
```

Replace the existing backend address with the public Render Web Service address and commit the change.

Next, open:

```
server/index.js
```

Locate the allowed frontend origin in the server configuration. Change it so that it corresponds to the GitHub Pages address associated with your GitHub username and forked repository. Commit the change.

A.5 GitHub Pages Deployment

Open the **Actions** tab in GitHub and enable workflows if prompted.

Then go to:

Settings → Pages

Under the deployment source, select:

GitHub Actions

Return to the **Actions** tab, select the **Deploy GitHub Pages** workflow, and run it. When the workflow finishes successfully, the frontend website will be available through GitHub Pages.

A.6 Keeping the Render Service Active

Render's free Web Service may wind down when it is inactive. The repository includes a GitHub Actions workflow that periodically contacts the backend.

In the repository, open:

```
.github/workflows/keep-alive.yaml
```

Replace the existing service domain with the public domain copied from Render. Preserve the health-check path:

```
/api/health
```

Commit the change. The workflow will periodically contact the health endpoint to reduce delays caused by the free service winding down.

A.7 Database Initialization

A newly created database is completely blank and must be initialized once before the website can be used.

First, open the PostgreSQL database in Render and copy its **External Database URL**. This differs from the Internal Database URL used by the Render Web Service.

In the GitHub repository, go to:

Settings → Secrets and variables → Actions

Create a new repository secret named:

```
DATABASE_URL
```

Paste the Render **External Database URL** as the secret value.

Next, return to the GitHub **Actions** tab, select the **Initialize Database** workflow, and run it. The workflow creates the required tables, accounts, events, and initial ticket inventory.

The initialization workflow is required only when the database is completely blank. Instructors do not need to use `curl`, ReqBin, or a `SEED_SECRET` to initialize it.

After initialization, the administrator can use the **Reset Database** button in the admin panel when a classroom reset is needed.

A.8 Creating Classroom Accounts

In the GitHub repository, open:

`server/users.js`

Table 14: SeatGate Classroom Account Roles

Account	Role	Classroom Purpose
`admin`	Administrator	Resets the database, releases tickets, and reviews evidence
`instructor`	Instructor	Demonstrates the automated five-ticket purchase
`fake-purchase`	Simulated user	Creates an intentional internal-control issue
`student01--student50`	Students	Participate in the trial and main ticket events

The repository includes approximately 50 default student accounts. These accounts initially use a common password and should be reviewed before the class.

Do not replace the usernames with real student names, because the repository may be publicly visible. Accounts should instead use anonymous identifiers such as:

`student01`, `student02`, `student03`, ...

Instructors may use an AI tool to generate unique passwords for the anonymous usernames. However, they should not submit the class roster or other identifiable student information to a commercial AI service.

The generated usernames and passwords can be copied into a private spreadsheet and matched with the instructor's class list locally. The spreadsheet can then be shown or distributed securely during class.

The account list should retain the following special accounts:

- one administrator account;
- one instructor account;
- one fake-buyer account used as part of the internal-control discussion; and
- the anonymous student accounts.

After changing the account list, commit the changes and run the appropriate deployment and database-initialization steps again if a new blank database is being used.

A.9 Opening and Checking the Website

After the GitHub Pages deployment finishes, go to:

Settings → Pages

Open the published website address.

Log in with the administrator username to confirm that the admin panel loads correctly. The panel should display the users, tickets sold, revenue, trial-ticket availability, main-ticket availability, ticket inventory, and user ticket holdings.

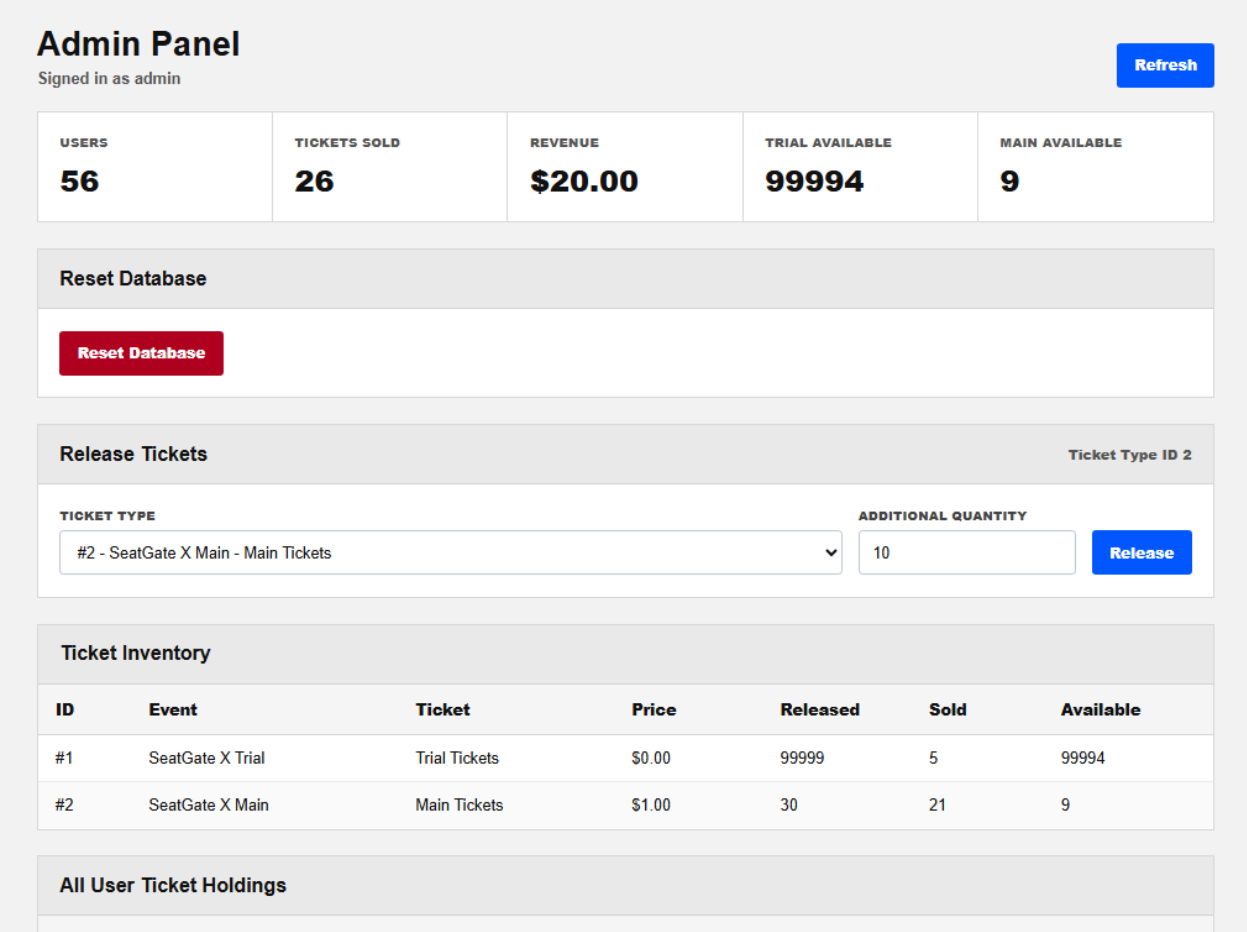

Figure 6: SeatGate administrator panel used to manage ticket releases and review user holdings

Figure 6 shows the administrator view. The dashboard summarizes the number of users, tickets sold, revenue, and the remaining trial and main ticket inventory. It also allows the administrator to reset the classroom database, release additional tickets, and review the ticket holdings of all users. During the debrief, these records provide evidence for reconciling purchases with inventory and identifying unusual holdings.

Before the classroom activity, the administrator may reset the database and release the required number of tickets through the admin panel.

A.10 Instructor Demonstration

Log in using the instructor account. The instructor page includes an **Automated Buy** button.

The button demonstrates that an automated request can buy five tickets in one transaction, even when the normal student interface is designed to restrict ordinary purchases. The demonstration remains capped at five tickets for classroom safety.

After selecting **Automated Buy**, return to the admin panel and confirm that:

- the instructor's ticket holdings increased;
- the number of tickets sold increased;
- the available inventory decreased; and
- the transaction is visible in the administrative evidence.

The instructor page also displays pseudocode representing the automated sequence:

```
token = SIGN_IN("instructor", "password")

ticket = GET("/api/events/1/tickets").tickets[0]

POST("/api/purchase",
    headers = {
        "Authorization": "Bearer " + token
    },
    body = {
        "eventId": 1,
        "items": [
            {
                "ticketTypeId": ticket.id,
                "quantity": 5
            }
        ]
    }
)
```

The instructor can explain that the browser sends requests to the backend when a user signs in, retrieves ticket information, and confirms a purchase. The same request sequence can be identified through the browser's developer tools and reproduced by an automated program.

A.11 Classroom Debrief

After the ticket activity and instructor demonstration, students should evaluate whether the stated controls were properly designed and whether they operated effectively.

The instructor can use the resources provided. Continue showing the second part of the video https://youtu.be/hIh2D8scudc to debrief the control failures and another video https://youtu.be/u1Z670PWvwY on how to

The discussion should address:

- whether the one-ticket purchasing restriction was enforced by the backend;
- whether only valid users could access the system;
- whether ticket inventory reconciled with completed purchases;
- whether automated purchases were prevented or merely recorded;
- whether exception logs were actively reviewed; and
- what management should change before using the system in production.

Students should support each conclusion with evidence such as screenshots, transaction results, inventory changes, ticket holdings, or log entries.

The main lesson is that frontend restrictions alone are not sufficient. Important controls must be enforced by the backend and supported by authorization rules, transaction limits, logging, exception reporting, and active monitoring.